Python Common Services

Table of Contents

Copyright Notice

Change History

1.0	Initial Version
2.0	Revised package strutcure following transport abstraction and ORCA package generation.

1. Introduction

PyCS is a Python API and framework to assist with writing Python clients that connect to the Common Services network.

The framework provides a number of packages which implement the different layers of the connection.

The diagram below illustrates the dependencies between the PyCS framework packages and the generated microformat and message bindings required to support them.

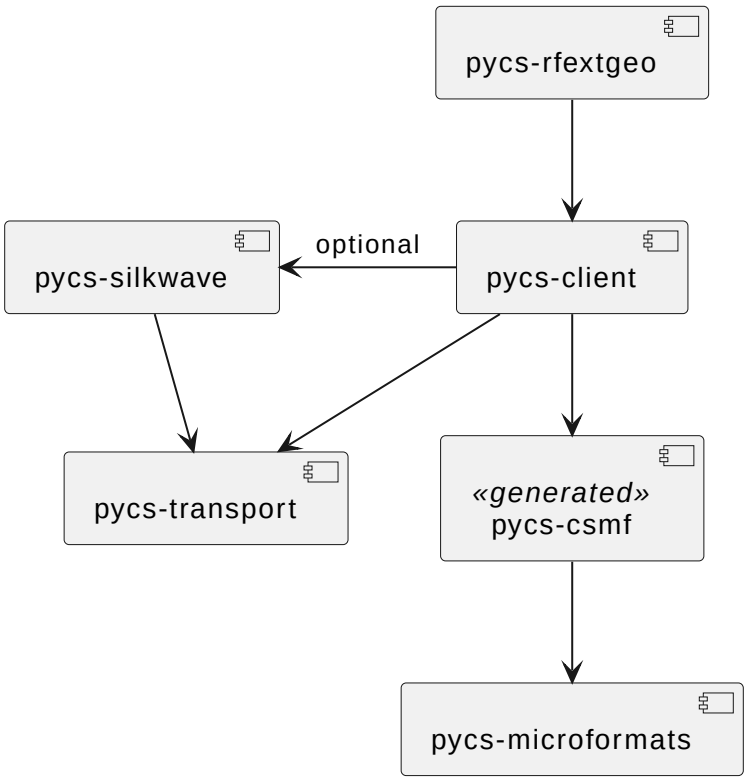

Figure 1. Package Hierarchy

1.1. Microformat and Message Bindings

The `pycs-microformats` package provides tools to allow generation of Python bindings for Microformats and Messages from their XML definitions. Some of these bindings are used by other packages in the framework, and must therefore be generated using the conventions expected by these packages. The `pycs.csmf` package contains all the definitions required by the other core packages. See [installation] for more details on generating these bindings.

1.2. Transport

The `pycs-transport` package provides an abstraction of the facilities needed by a client to connect to a Common Services network, with services to handle connection to the network, sending and receiving messages, file transfers and UDP/RTP streaming.

1.3. SILKWAVE Connection

The `pycs-silkwave` package provides an implementation of the transport enabling connection to a SILKWAVE network, with additional services to handle SILKWAVE network ping requests.

1.4. Common Services Client

The `pycs-client` package provides a facilities to handle connection to the Common Services network. This layer provides an interface to the Registration Service, mangaging registration of clients onto the network. It also allows clients to publish Capability and Status messages to the network, and to subscribe for updates from other clients. Signing and verification of messages is also supported.

1.5. RfExtGeo

The `pycs-rfextgeo` package provides utilities for handling RfExtGeo specific PSD and audio streams over RTP, and for creating and reading RfExtGeo Midas Blue files. It also provides basic example applications for both a dummy RfExtGeo sensor and command line tasker.

2. Installation

The documentation for the individual PyCS framework packages give details on how to install those packages, but assume that a pypi-style repository is available containing all of the dependencies. Where these packages are the generated microformat and message bindings, these will often have been published to a local repository by CI/CD processes or admins. If this is not the case, for example when using pre-release definitions, or in isolated development environments, these must be generated manually from the XML definitions and installed into your virtual evnironment or published to a local pypi repository.

The PyCS core packages require a `pycs.csmf` package generated from an ORCA repository containing the core microformat and message definitions.

2.1. Generating Microformat and Message packages

For all of the commands below, a python virtual environment should be used with the `pycs-microformats`, `build` and `twine` packages installed.

```
$ python -m venv venv
$ ./venv/bin/activate
(venv)$ pip install pycs.microformats build twine
```

2.2. Generating binding code

2.2.1. Generating from an ORCA repository.

The easiest way to generate all the required packages is from an ORCA repository. An ORCA repository contains all the message and microformat definitions required, along with artifact definitions detailing which messages and/or microformats should be contained in each package.

The stucture of an ORCA repository is as follows:

The following command can be used to generate a binding package for all artifacts contained within the orca repository. The resultant bindings will be generated to the 'generated/<package>/<version>' directory, ready to be uploaded to a repository. The orca-generate command has options to allow generation with a different package name for non-core packages.

```
(venv)$ orca-generate
```

2.2.2. Generating from individual definition directories

If an ORCA repository is not available, the process for generating the required packages is significantly more complicated, and requires each package to be generated separately using the `pycs-generate` command. Each package must be generated using the installation package name and python module names that the PyCS framework expects, and must specify any required packages and imports explicitly. These dependency packages must be installed and available in the virtual environment before generating each package.

The tables in the following section detail the AOCO numbers, package and module names for each package that the framework expects, and the dependencies required by each one. The dependencies are correct at the time of writing, but may be subject to change as the definitions change.

Each microformat package should be generated with the following command line, where `<version>`

is the version number of the package to generate, `<path>` is the path containing the definition files. The `--imports` and `--requires` arguments should be repeated for each dependency as appropriate.

To ensure that the definitions are available for generation of subsequent packages, the recently generated package should be imported into the virtual environment after each generation step.

NOTE Using this method to generate packages will create a slightly different package structure to the ORCA method, as extra packages are required to store the definitions for collections and extension base classes and make them available to subsequent generation steps. These extra packages are not required by the ORCA method, as all definitions are visible at generation time.

```
(venv)$ pycs-generate --package aoco-<aoco>-<name>-domain \
                      --version <version> \
                      --mf-module csmf.mf.<module> \
                      --imports <import> \
                      --requires <requires> \
                      --output generated/aoco-<aoco>-<name>-domain/<version> \
                      <path>
(venv)$ pip install -e generated/aoco-<aoco>-<name>-domain/<version>
```

Each message package should be generated with the following command line:

```
(venv)$ pycs-generate --package aoco-<aoco>-<name>-messages \
                      --version <version> \
                      --imports <import> \
                      --requires <requires> \
                      --output generated/aoco-<aoco>-<name>-messages/<version> \
                      <path>
(venv)$ pip install -e generated/aoco-<aoco>-<name>-messages/<version>
```

2.3. Uploading to a repository

Once the bindings have been generated, they can be built into python wheel files and uploaded to a pypi repository for future use.

```
(venv)$ for p in generated/*/*/pyproject.toml; do python -m build --wheel $(dirname $p); done
(venv)$ twine upload --repository-url $REPOSITORY generated/*/*/*.whl
```